

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

Τεχνολογίες Υπηρεσιών Λογισμικού

Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

Data layer technologies: ORM

Χρίστος Χατζηχριστοφή, υ.δ.ΕΜΠ

Prerequisites

- Basic knowledge of Node.js and JavaScript
- Basic knowledge of Relational Databases and SQL

Introduction to ORMs

- Why use an ORM?
- How does an ORM work?
- Examples of popular ORMs
- Benefits of using Sequelize
- Conclusion

Why use an ORM?

- Object-Relational Mapping (ORM) is a technique used to map data between an application and a database
- It provides an abstraction layer that allows developers to interact with a database using objects, rather than SQL queries

How does an ORM work?

- ORMs typically use a mapping layer that maps between object-oriented concepts (such as classes and objects) and relational database concepts (such as tables and columns)
- The ORM translates object-oriented commands into SQL commands and maps the result sets back to objects
- The ORM provides a set of APIs that developers can use to perform common database operations

Examples of popular ORMs

- Hibernate (Java) 
- Entity Framework (C#) 

- Django ORM (Python) 

- Sequelize (JavaScript/Node.js) 

Sequelize for JS ecosystems

- Popular ORM library for Node.js and JavaScript applications
- Provides a robust set of APIs for performing common database operations, such as querying, updating, and deleting data
- Supports a wide range of databases, including MySQL, PostgreSQL, and SQLite
- Provides built-in support for features such as migrations, associations, and validations, making it easy to maintain and evolve your database schema

In brief

- ORMs provide a powerful abstraction layer that simplifies database interactions and reduces boilerplate code
- Sequelize is a popular ORM library for Node.js that provides a wide range of features and supports a variety of databases
- By using Sequelize, developers can focus on writing high-level business logic, rather than worrying about low-level database interactions.

A walk through Sequelize

Introduction to Sequelize

- What is Sequelize
- Why use an ORM like Sequelize?
- Installing and Setting up Sequelize
- Defining a Model in Sequelize
- Using Sequelize APIs

What is Sequelize

- Sequelize is an Object-Relational Mapping (ORM) library for Node.js that allows you to interact with databases using JavaScript objects
- Sequelize provides a robust set of APIs for performing common database operations, such as querying, updating, and deleting data
- Sequelize supports a wide range of databases, including MySQL, PostgreSQL, and SQLite

Why use an ORM like Sequelize?

- ORMs simplify database interactions by abstracting away the complexities of SQL and database-specific syntax
- ORMs reduce the amount of boilerplate code required to perform common database operations, such as CRUD (Create, Read, Update, Delete) operations
- ORMs can improve code maintainability, as they allow developers to work with a higher-level object-oriented abstraction, rather than directly manipulating the database schema

Installing and Setting up Sequelize

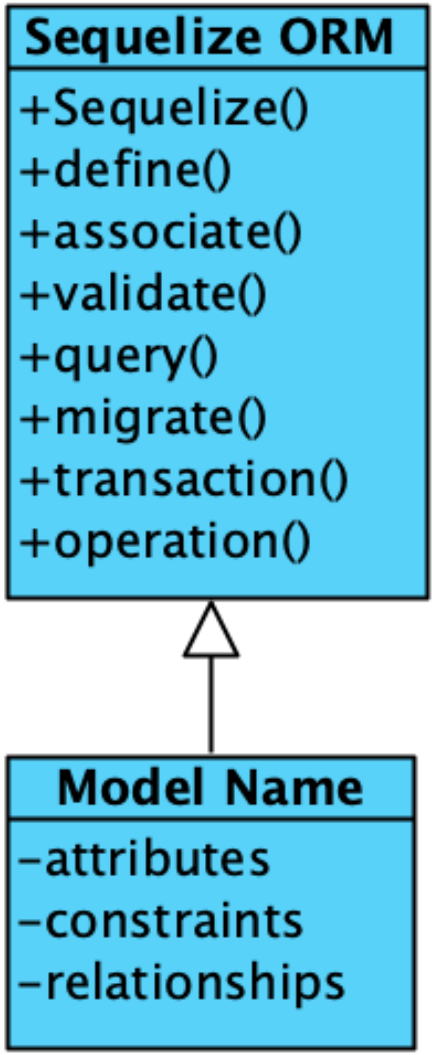
- To use Sequelize, you'll first need to install the package using npm or yarn
- Once you've installed Sequelize, you'll need to create a connection to your database by defining a configuration object with your database credentials
- You'll also need to define a Model in Sequelize, which represents a table in your database and provides a high-level abstraction for working with data

Defining a Model in Sequelize

- In Sequelize, Models are defined using a class that extends the Sequelize Model class
- Models define the table name, column names, data types, and relationships between tables
- Sequelize provides a variety of built-in data types, such as STRING, INTEGER, and BOOLEAN, as well as support for custom data types

Models inherit Sequelize ORM's operations

- Model definition: Define models that represent tables
- Data validation: Mechanisms for validating model attributes
- Querying: Powerful query interface to perform complex queries
- Associations: One-to-One, One-to-Many, Many-to-Many
- Migrations: Manage schema updates, keeps your db in sync with your model definitions
- Transactions: Perform multiple operations as a single unit of work



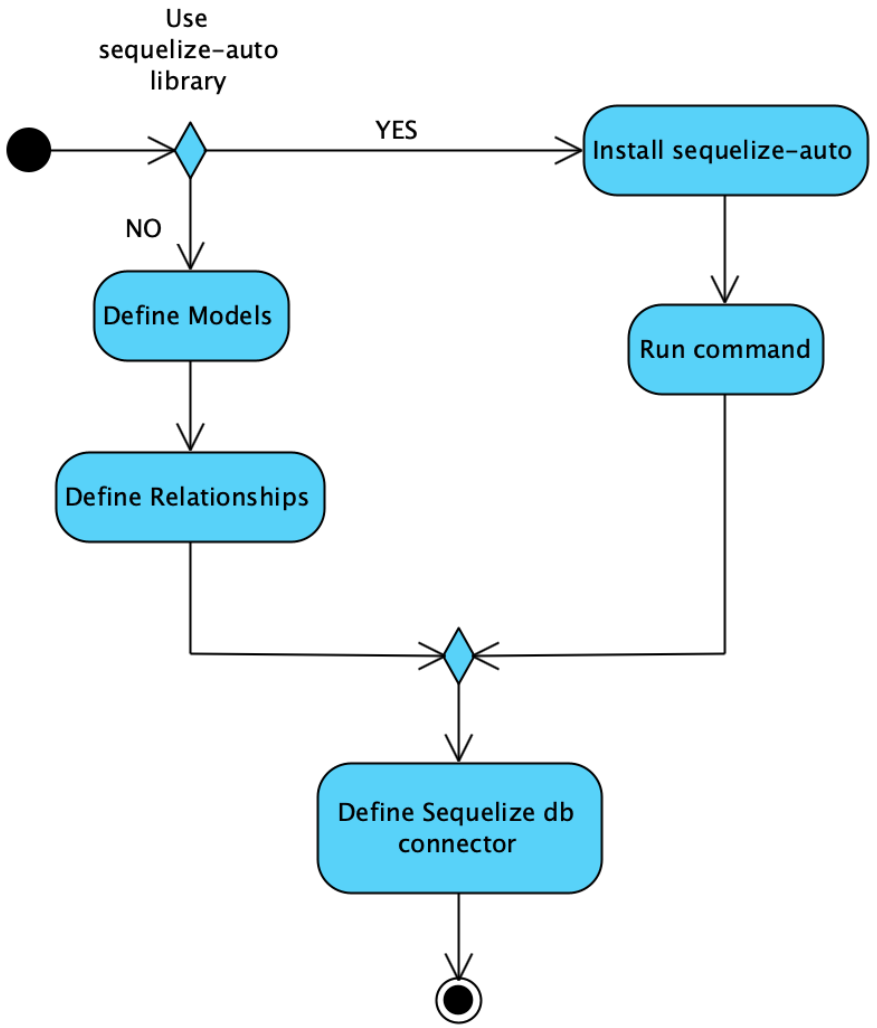
Using Sequelize APIs

- Sequelize provides a wide range of APIs for performing common database operations, such as querying, updating, and deleting data
- Sequelize APIs use a fluent syntax, which allows you to chain multiple commands together in a single line of code
- Sequelize also provides support for advanced features such as transactions, associations, and validations

Example methods of Sequelize Model API

public static	async `create(values: object, options: object): Promise<Model>` Builds a new model instance and calls save on it.
public static	async `decrement(fields: string | Array | object, options: object): Promise<Model[], ?number>` since 4.36.0 Decrement the value of one or more columns.
public static	async `describe(schema: string, options: object): Promise` Run a describe query on the table.
public static	async `destroy(options: object): Promise<number>` Delete multiple instances, or set their deletedAt timestamp to the current time if `paranoid` is enabled.
public static	async `drop(options: object): Promise` Drop the table represented by this Model
public static	async `findAll(options: object): Promise<Array<Model>>` Search for multiple instances.
public static	async `findAndCountAll(options: object): Promise<{count: number|number[], rows: Model[]}>` Find all the rows matching your query, within a specified offset / limit, and get the total number of rows matching your query.
public static	async `findByPk(param: number | bigint | string | Buffer, options: object): Promise<Model>` Search for a single instance by its primary key._
public static	async `findCreateFind(options: object): Promise<Model, boolean>` A more performant findOrCreate that may not work under a transaction (working in postgres) Will execute a find call, if empty then attempt to create, if unique constraint then attempt to find again
public static	async `findOne(options: object): Promise<Model|null>` Search for a single instance.
public static	async `findOrBuild(options: object): Promise<Model, boolean>` Find a row that matches the query, or build (but don't save) the row if none is found. The successful result of the promise will be (instance, built)
public static	async `findOrCreate(options: object): Promise<Model, boolean>` Find a row that matches the query, or build and save the row if none is found The successful result of the promise will be (instance, created)

How to integrate Sequelize ORM



Defining Models

- A Model in Sequelize represents a database table and provides a high-level abstraction for working with data
- Sequelize provides a simple syntax for defining Models, including specifying attributes, data types, and options

Defining Model Attributes

- Attributes in Sequelize correspond to table columns in the database
- Attributes are defined using an object literal, with the key representing the column name and the value representing the data type
- Sequelize supports a wide range of data types, including STRING, INTEGER, BOOLEAN, DATE, and more

Defining Model Options

- Options in Sequelize provide additional configuration for Models, such as specifying table names, timestamps, and associations
- Options are defined using an object literal, and can be passed as the second argument to the define() method when defining a Model

Generate SQL Statements from Model Definitions

- Sequelize generates SQL statements based on Model definitions, allowing you to interact with the database using JavaScript objects
- When you perform a query using a Model, Sequelize generates the appropriate SQL statement based on the Model definition

Creating Instances of a Model

- To create a new instance of a Model, you can call the create() method on the Model class and pass in an object with the values for the Model attributes
- Sequelize will automatically generate the appropriate SQL statement to insert the new instance into the database

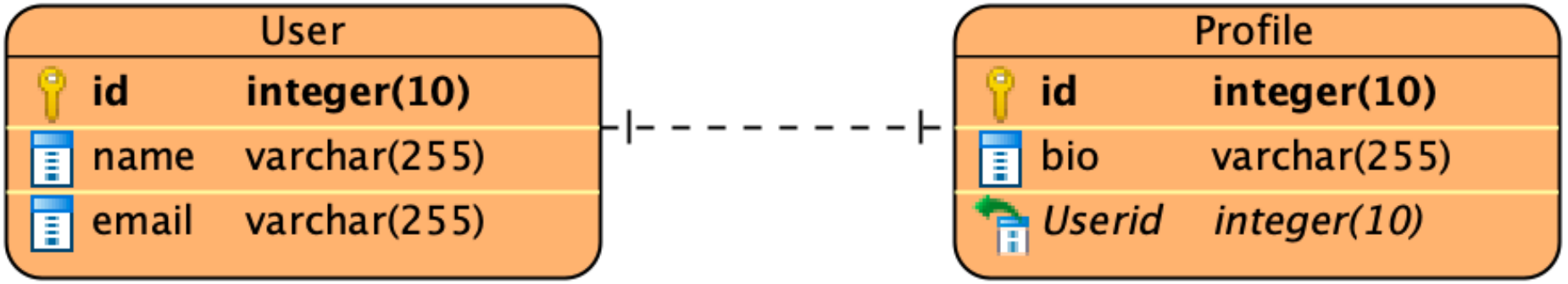
Associations between Models

- Associations define relationships between Models, such as one-to-one, one-to-many, and many-to-many relationships
- Sequelize provides a simple syntax for defining Associations between Models, including specifying the type of Association, the foreign key, and any additional options

Defining One-to-One Associations

- A one-to-one Association is used to define a relationship where one instance of a Model is associated with exactly one instance of another Model
- In Sequelize, you can define a one-to-one Association by calling the hasOne() method on the source Model and passing in the target Model as an argument

Example: One to One ER Diagram



Example: One to One Model Definition

Profile Model with a one-to-one association to User Model:

- A Profile belongs to a User, while a User has one Profile

```
// User model definition
const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true
  },
  name: {
    type: DataTypes.STRING
  },
  email: {
    type: DataTypes.STRING
  }
});

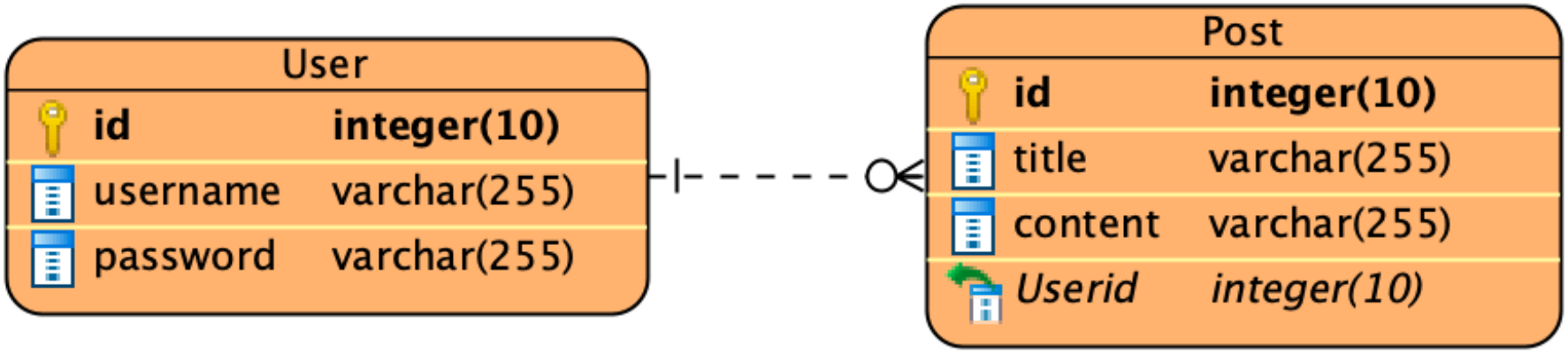
// Profile model definition
const Profile = sequelize.define('Profile', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true
  },
  bio: {
    type: DataTypes.TEXT
  }
});

// Establishing the one-to-one association
User.hasOne(Profile, { foreignKey: 'user_id' });
Profile.belongsTo(User, { foreignKey: 'user_id' });
```

Defining One-to-Many Associations

- A one-to-many Association is used to define a relationship where one instance of a Model is associated with many instances of another Model
- In Sequelize, you can define a one-to-many Association by calling the hasMany() method on the source Model and passing in the target Model as an argument

Example: One to Many ER Diagram



Example: One to Many Model Definition

User Model with a one-to-many association to Post Model:

- The User Model has many Posts, while a Post belongs to a User

```
// User Model
const User = sequelize.define('User', {
  id: {
    type: DataTypes.INTEGER(10),
    primaryKey: true,
    autoIncrement: true
  },
  username: DataTypes.STRING,
  password: DataTypes.STRING
});

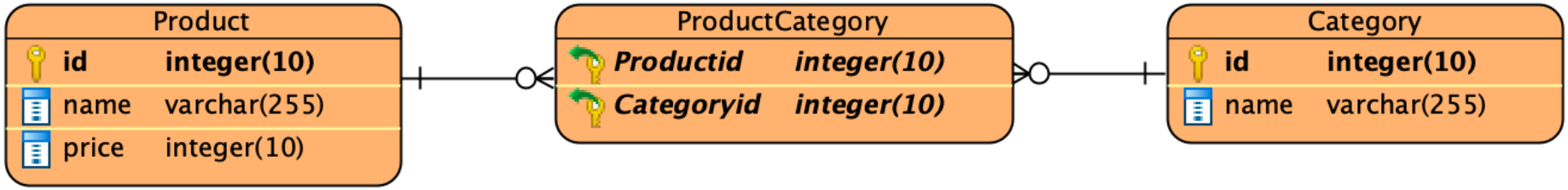
// Post Model
const Post = sequelize.define('Post', {
  id: {
    type: DataTypes.INTEGER(10),
    primaryKey: true,
    autoIncrement: true
  },
  title: DataTypes.STRING,
  content: DataTypes.STRING
});

// One-to-many Association
User.hasMany(Post);
Post.belongsTo(User);
```

Defining Many-to-Many Associations

- A many-to-many Association is used to define a relationship where many instances of a Model are associated with many instances of another Model
- In Sequelize, you can define a many-to-many Association by defining a third Model, known as a join table, that maps the relationship between the two Models

Example: Many to Many ER Diagram



Example: Many to Many Model Definition

Product Model with a many-to-many association to Category Model:

- A Product can belong to many Categories, while a Category can have many Products

```
// Product Model
const Product = sequelize.define('Product', {
  id: {
    type: DataTypes.INTEGER(10),
    primaryKey: true,
    autoIncrement: true,
  },
  name: DataTypes.STRING,
  price: DataTypes.INTEGER
});

// Category Model
const Category = sequelize.define('Category', {
  id: {
    type: DataTypes.INTEGER(10),
    primaryKey: true,
    autoIncrement: true,
  },
  name: DataTypes.STRING
});

// Many-to-many Association
Product.belongsToMany(Category, { through:
'ProductCategory' });
Category.belongsToMany(Product, { through:
'ProductCategory' });
```

Validations in Sequelize

- Validations are a way to ensure data integrity and consistency in your database
- Sequelize provides a variety of built-in validators that you can use to validate data before it's saved to the database
- You can also create custom validators to meet your specific validation needs

Built-in Validators

Sequelize provides a number of built-in validators that you can use to enforce constraints on your data, such as:

- required: ensures that a field is not null or undefined
- unique: ensures that a field has a unique value
- min/max: ensures that a numeric field is within a certain range
- len: ensures that a string field has a minimum and/or maximum length

Using Built-in Validators

- To use a built-in validator in Sequelize, you simply need to include it as an option when defining a Model attribute
- For example, to make a field required, you would add the 'allowNull: false' option to the attribute definition
- To make a field unique, you would add the 'unique: true' option to the attribute definition

Custom Validators

- Sequelize also allows you to create your own custom validators to meet your specific validation needs
- Custom validators are functions that take a value as an argument and return either true or false depending on whether the value is valid
- You can add a custom validator to a Model attribute by defining it as a function and including it as an option in the attribute definition

Handling Validation Errors

- When a validation fails, Sequelize throws a ValidationError object, which contains details about the failed validation(s)
- You can handle validation errors in your code by catching the ValidationError and handling it appropriately
- Common ways to handle validation errors include logging the error, displaying an error message to the user, or rolling back a transaction

Validation Errors Conclusion

- Validations are an important part of ensuring data integrity and consistency in your database
- Sequelize provides a robust set of built-in validators, as well as the ability to create custom validators, to meet your validation needs
- By handling validation errors appropriately, you can ensure that your application behaves predictably and consistently.

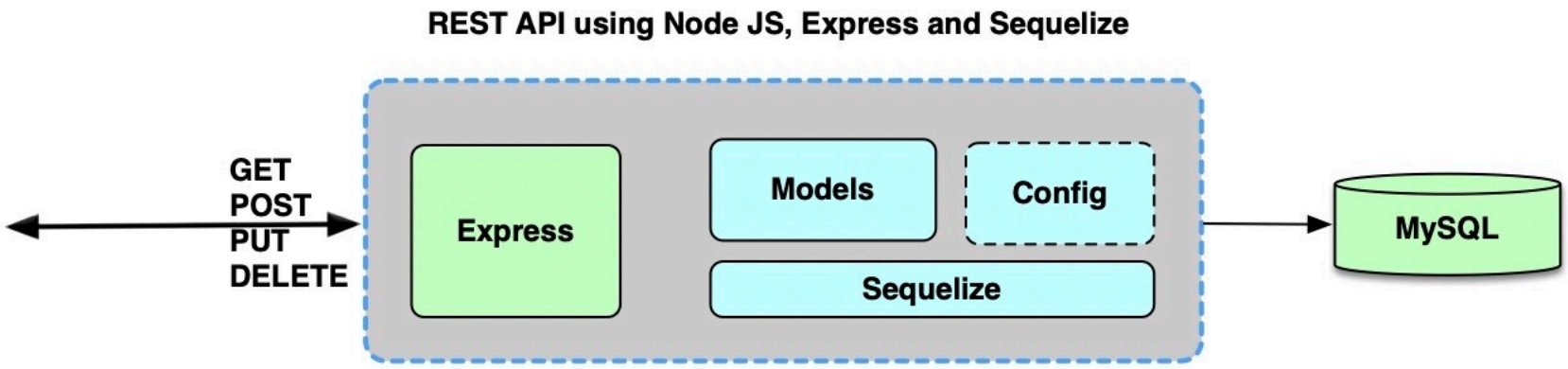
Example 1 – Using Built-in Validation

```
const User = sequelize.define('User', {
  firstName: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notNull: { msg: 'Please enter your first name' },
      notEmpty: { msg: 'Please enter your first name' }
    }
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    unique: true,
    validate: {
      notNull: { msg: 'Please enter your email address' },
      notEmpty: { msg: 'Please enter your email address' },
      isEmail: { msg: 'Please enter a valid email address' }
    }
  }
});
```

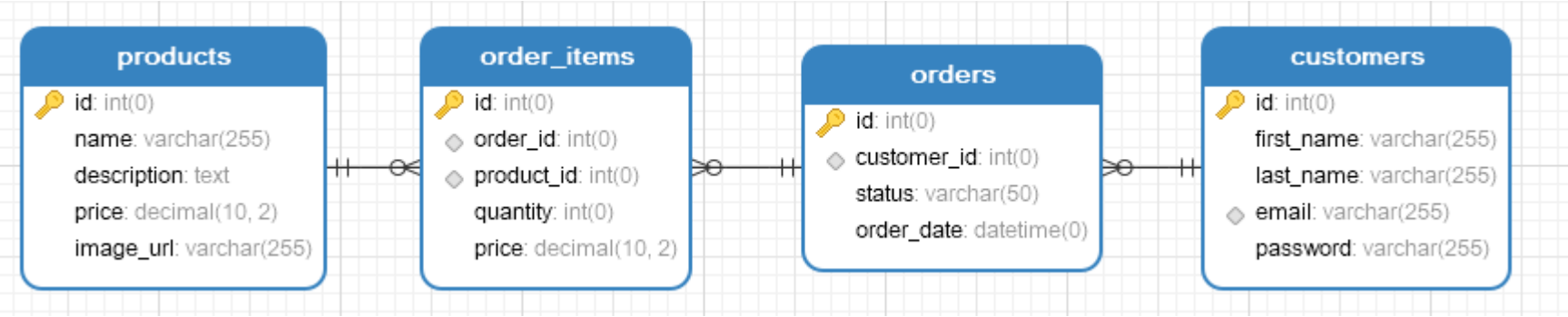
Example 1 – Using Custom Validation

```
const User = sequelize.define('User’, {
  age: {
    type: DataTypes.INTEGER,
    validate: {
      isEven(value) {
        if (value % 2 !== 0) {
          throw new Error('Age must be an even
number’);
        }
      }
    }
  }
});
```

Rest API (NodeJS + Express), Sequelize, MySQL



HANDS-ON



- Querying Data
- Updating and Deleting Data

Useful Links

- Sequelize ORM
- Sequelize npm
- NodeJS
- Sequelize-auto npm
- Sequelize Demo GitHub repo

Thanks for your attention!

